1 Summary

I present an off-policy deep reinforcement learning (DRL) algorithm for a class of queueing networks referred to as sharing networks. This setting involves a countable state space and a continuous action space given by a system of linear inequalities. However, it can be shown that at least one of the vertices of the action space is always optimal, meaning we can restrict our action search to this discrete set. In this setting transition probabilities are only influenced by the state when at a boundary state. This makes it possible to pool transitions from all states and fit an accurate model of these probabilities with a relatively small sample size. I used a Double DQN-type approach (Hasselt, Guez, and Silver), but this transition probability model allows us to write the Q function in a restricted form, making it more efficient to estimate. In addition, the transition probability model allows us to work with "conditional" versions of value estimates which should further improve sample efficiency.

2 Sharing Networks

Sharing networks were introduced by Massoulié and Roberts as models for internet traffic. A sharing network is a queueing network where there are J different job types, each with its own queue with infinite buffer, and I resources for prcoessing them, where $I < J$. I consider the case where job arrivals to each queue are given by a Poisson process and job sizes are i.i.d. exponential random variables. The control is a processing rate assigned to each job type, and when the integrated processing rate assigned to a job reaches the its size the job is completed and leaves the network. A crucial feature of sharing networks is that processing certain job types requires simultaneous bandwidth form multiple resources. This "sharing" behavior is captured by an $I \times J$ incidence matrix K, where $K_{i,j} = 1$ if processing job type j requires bandwidth from resource i, and $K_{i,j} = 0$ otherwise. Each resource has a corresponding bandwidth capacity constraint, given by an I-dimensional positive vector $C = (C_1, ..., C_I)$. Consequently, any admissible processing rate control, $a = (a_1, ..., a_J)$, which corresponds to I-dimensional bandwidth usage vector Ka, must satisfy the bandwidth capacity constraint $Ka \leq C$ component-wise.

For an example of a simple sharing network see the illustration of a *Two-Link Linear Network* (2LLN) in figure 2.1. There are $J = 3$ job types and $I = 2$ resources for processing them. The first two jobs types ($j = 1, 2$) only require a single resource for processing. These are referred to as *local traffic* jobs. In contrast, job type 3 requires all resources for processing, which I will refer to as the *shared traffic* job. This processing dependence structure is described by the incidence matrix

$$K = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

The mean interarrival times and job sizes can vary based on job type. For job type j arrivals are given by a Poisson process with rate α_j and job sizes are exponentially distributed with rate β_j. Mutual independence is assumed. Note that since job sizes are exponential the particular service policy, such as first-in first-out, has no impact on the probability distribution of the process. Let $\rho \doteq \alpha/\beta$ and observe that an action of $a = \rho$ results in jobs being completed at the exact rate at which they're arriving. As a result $C > K\rho$ is required for stability, and considering a sequence where $K\rho \uparrow C$ is referred to as a heavy traffic limit. In all of the examples presented here $C_i = 1$ and $(K\rho)_i = .8$ for all i.

Let $S(t) = (S_1(t), ..., S_J(t))$ represent the queue lengths at time t, and since each queue length is a nonnegative integer the state space is

$$\mathcal{S} \doteq \mathbb{Z}_+^J.$$

Due to the fact that processing rates must be nonnegative and satisfy the capacity constraint the corresponding action space is

$$\mathcal{A} \doteq \left\{ a \in \mathbb{R}_+^J : Ka \leq C \right\}.$$

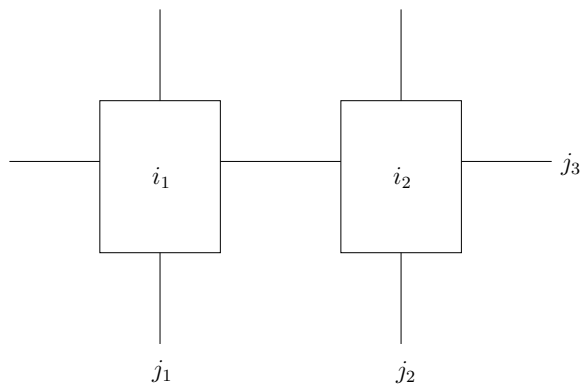

Figure 2.1: 2LLN Network

Observe that, since we can't process jobs in an empty queue, we could impose the additional action constraint that $a_j(t) = 0$ if $S_j(t) = 0$. For simplicity I allow $a_j(t) > 0$ when $S_j(t) = 0$ but define the corresponding *flowrate* to be $x_j(t) = a_j(t)\mathcal{I}_{\{S(t)>0\}}$. The flowrate is simpy the actual processing rate that results from an action, after accounting for the fact that empty queues can't be processed.

We can write the queue length as

$$S_j(t) = N_j^A(t) + N_j^C(X_j(t))$$

where $X_j(t) = \int_0^t x_j(r)dr$ is the integrated flowrate assigned to type j jobs by time t, $N_j^A(\cdot)$ is a Poisson process with rate α_j, and $N_j^S(\cdot)$ is a Poisson process with rate β_j. Here $N_j^A(t)$ gives the number of (type j) jobs that arrived by time t and $N_j^C(X_j(t)$ gives the number of type j jobs that have been completed by time t. Of course any reasonable action policy must satisfy certain nonanticipatory conditions (i.e. the system adminstrator can't know the specific times of future arrivals). Here I focus exclusively on actions which are a function of the state and I use $\pi : \mathcal{S} \to \mathcal{A}$ to represent this action mapping. Observe that this means the action cannot depend on the sizes of the jobs currently in the queue, so these can be thought of as unknown quantities.

Shorter queues are preferable (less wait time for customers, less storage space required, etc.) which is quantified by a nonnegative holding cost function $h : \mathcal{S} \to \mathbb{R}_+$. The form of this function is flexible, but it is assumed that any positive queue has a cost ($h(s) > 0$ unless $s = 0$) and longer queues are more costly than shorter queues ($h(\tilde{s}) \geq h(s)$ if $\tilde{s} \geq s$ component-wise). Our goal is to minimize the infinite horizon discounted cost

$$E\left[\int_0^\infty e^{-\kappa} h(S(t))dt\right]$$

for some discount factor κ. In the examples considered here I will use linear holding costs.

A quantity of interest is the resource workload, which is the total work (integrated bandwidth) required from each resource to complete all of the jobs currently in the queue. The workload at time t is the I-dimensional process

$$W_i(t) = \sum_j K_{i,j}\left(\sum_k^{N^{A_j(t)}} \eta_j(k) - X_j(t)\right),$$

where $\eta_j(k)$ is the size of the kth job that has arrived of type j. A challenging feature of sharing networks is *blocking*, where the processing of certain job types can prevent resources from working at full capacity despite having positive workloads (see Section 5.1 for an example of this in a 2LLN network). I refer to resources with positive workloads operating under their capacity constraint as *wasted bandwidth*. This

can be thought of as a processing opportunity cost which can lead to future increased holding costs. When selecting actions it is important to think about wasted bandwidth in addition to the rate at which that action reduces the current holding cost.

We can write the workload as

$$W_i(t) = B_i(t) + \xi_i(t)$$

where

$$B_i(t) = \sum_j K_{i,j} \left(\sum_k^{N^{A_j(t)}} \eta_j(k) \right) - tC_i$$

and

$$\xi_i(t) = tC_i - \sum_j K_{i,j} X_j(t)$$

is the total unused bandwidth of resource i at time t. This representation has the advantage that $B_i(t)$ has no dependence on the control and $\xi_i(t)$ is a nonnegative, nondecreasing process which must satisfy $B_i(t) + \xi_i(t) \geq 0$ for all t. Consequently, for any admissible control, we must have $\xi_i(t) \leq \tilde{\xi}_i(t)$ for all t, where

$$\tilde{\xi}_i(t) \doteq -(0 \wedge \inf_{s \leq t} B_i(s)).$$

Note that $\tilde{\xi}_i(t)$ corresponds to a policy with no wasted bandwidth (which, depending on the sharing network, is not always possible). It follows that $W_i(t) \geq \tilde{W}_i(t)$ for all t, where

$$\tilde{W}_i(t) \doteq B_i(t) + \tilde{\xi}_i(t).$$

This provides a lower bound for the workload process, but in order to use it as a benchmark we need to connect it to the holding cost which defined in terms of the queue length. Note that for any queue length $S(t) \in \mathbb{Z}_+^J$ with holding cost $h(S(t))$ (due to the assumption of exponentially distributed job sizes) the conditional expected workload is

$$E\left[W(t)|S(t)\right] = \sum_j K_{i,j} S_j(t)/\beta_j.$$

We define the workload holding cost as

$$\hat{h}(w) \doteq \min \left\{ h(s) : s \in \mathbb{R}_+^J \text{ and } w_i = \sum_j K_{i,j}(s_j/\beta_j) \text{ for all } i \right\},$$

which, due to the above conditional expectation, can be thought of as the minimum holding cost of all queue lengths whose expected workload is w. Note that in this definition s doesn't have to be integer valued ($s \in \mathbb{R}_+^J$ rather than $\mathbb{Z}_+^J$) and we assume $h(s)$ is defined on $\mathbb{R}_+^J$ in such a way that this infimum is not over an empty set (such as linear $h(s)$ which is used in the examples).

This gives rise to the workload formulation of the problem which is to minimize

$$E\left[\int_0^\infty e^{-\kappa t} \hat{h}(W(t)) dt\right].$$

Due to the discussion above this is bounded below by

$$E\left[\int_0^\infty e^{-\kappa t} \hat{h}(\tilde{W}(t)) dt\right]. \tag{2.1}$$

The connection between the workload formulation and original queue length formulation is most direct in the Brownian control problem that describes the asymptotic behavior of a sequence of these sharing

networks under a diffusion scaling in heavy traffic (Budhiraja, Conroy, and Johnson). In the Brownian control problem it can be shown that the two formulations are equivalent and the workload can be more convenient to work with (Harrison).

I introduced the workload formulation here to use (2.1) as a benchmark, which is the pre-asymptotic version of a performance criteria from the Brownian control problem called hierarchical greedy ideal (HGI) (Harrison, Mandayam, Shah, and Yang). The intuition behind HGI is a hierarchy of priorities. First minimize the workload (avoid wasting bandwidth), then be in a queue length configuration that minimizes the holding cost for the corresponding workload ($h(S(t)) = \hat{h}(W(t))$). It is important to note that while HGI performance is generally "good," it isn't necessarily optimal when $\hat{h}(w)$ is not a monotonically nondecreasing function of the workload (i.e. when there exists $\tilde{w} \geq w$ componentwise such that $\hat{h}(\tilde{w}) < \hat{h}(w)$). As a result I'll discuss the monotonicity of $\hat{h}(w)$ in the examples.

Here I focus on pre-asymptotic performance of an off-policy deep reinforcement learning algorithm. While I assume incidence matrix $K_{i,j}$ and capacity constraints C_i are known (so that the action space is known), the arrival and service rates (α_j, β_j) as well as the cost function $h(q)$ are unknown and must be learned by the algorithm. In the examples these unknown quantities $(\alpha_j, \beta_j, h(q))$ are fixed, but the algorithm is designed so that it should be able to adjust to a situation where they are changing (gradually).

3 Discrete Time Formulation

Let $\{\tau_i : i \in \mathbb{Z}_+\}$ denote the time of the ith jump of $S(t)$ (using the convention that $\tau_0 = 0$). In a slight abuse of notation let $S_i \doteq S(\tau_i)$ and observe that, under any state-dependent policy π, $\{S_i : i \in \mathbb{Z}_+\}$ is a Markov process and

$$\begin{aligned}\int_0^\infty e^{-\kappa t} h(S(t))dt &= \sum_{i=1}^\infty \int_{\tau_{i-1}}^{\tau_i} e^{-\kappa t} h(S(\tau_{i-1})dt \\ &= \sum_{i=1}^\infty \kappa^{-1} e^{-\kappa \tau_{i-1}} \left(1 - e^{-\kappa(\tau_i - \tau_{i-1})}\right) h(S_{i-1}).\end{aligned}$$

Consequently, we can treat this like a discrete-time reinforcement learning problem where the discount factor for future rewards depends on the action. Specifically, the action a_i is the constant control on the interval $[\tau_i, \tau_{i+1})$ (given by the state-dependent policy π), the future discount factor is $\gamma_i = e^{-\kappa(\tau_i - \tau_{i-1})}$, and the reward is

$$r_i = \kappa^{-1} \left(1 - e^{-\kappa(\tau_i - \tau_{i-1})}\right) (-h(S_{i-1})).$$

To be consistent with typical reinforcement learning conventions I treat the negative holding cost as a reward that we're trying to maximize. Our goal is then to maximize

$$-\int_0^\infty e^{-\kappa t} h(S(t))dt = \sum_{i=1}^\infty r_i \prod_{j=0}^{i-1} \gamma_j$$

where I use the convention that $\gamma_0 = 1$. Note that, the only part of the history of the process that $(r_{i+1}, \gamma_{i+1}, S_{i+1})$ depends on is (S_i, a_i). Specifically, if we let

$$\mathcal{G}_i \doteq \sigma\left(S_j, a_j, \gamma_j, r_j : j \leq i\right),$$

then for any $B \in \mathcal{B}((-\infty, 0] \times [0, 1) \times \mathbb{Z}_+^J)$ we have

$$P\left((r_{i+1}, \gamma_{i+1}, S_{i+1}) \in B | \mathcal{G}_i\right) = P\left((r_{i+1}, \gamma_{i+1}, S_{i+1}) \in B | S_i, a_i\right).$$

4 Reinforcement Learning Approach

Although the action space is continuous, I show that at least one of the vertices of the action space is optimal and restrict my action search to this finite set. This turns it into a discrete action space problem, and I use a Double DQN-type approach (Hasselt, Guez, and Silver) type approach. However, I write the state-action value function in terms of the state value function and the rate parameters (α, β) and estimate these separately with the intention of improving sample efficiency. In particular, the rate parameter estimates converge quickly using MLE which assists with fitting the neural network approximation of the state value function. Due to confidence in the parameter estimates, I use conditional n-step value estimates which results in a conditional TD(λ) (referred to as CTD(λ)) approach to computing target values when fitting the value function. This is all described in detail in this section.

I use an ϵ-greedy policy, and the "search" steps that occur with probability ϵ use action $a_j = \hat{\rho} \doteq \hat{\alpha}/\hat{\beta}$, where $(\hat{\alpha}, \hat{\beta})$ are the MLE estimates of (α, β). This is done to attempt a random walk action (i.e. increasing and decreasing the the queue lenght in each dimension are equally likely). Because this requires reasonable rate parameter estimates, I initially conduct a small number of batches using *random vertex actions*, which as the name suggests involves choosing a vertex action where all are given equal probability. Since the state space is unbounded (queues have infinite buffers), and initially poor actions could lead to extreme queue growth, I introduce a *cost bound* parameter when training the algorithm. If a state whose holding cost exceeds the cost bound is reached the trajectory is terminated and a final cost is imposed equal to incurring a holding cost given by the cost bound indefinitely into the future. As a result these terminal cost bound states are more costly than keeping the process active. Note that this truncated cost bound environment is only used in training. In the performance tests presented in Section **??** the regular unbounded state space is used.

Algorithm 1 DRL Algorithm

```
Initialize replay buffer B with fixed capacity
Initialize rate estimates (α̂, β̂) and V̂ MLP parameters
for m = 1, ..., M do
    if m ≤ N_rand then
        Simulate batch using random vertex action
    else
        Simulate batch using a ε_m-greedy action
            - chooses a = ρ̂ ≐ α̂/β̂ with probability ε_m
            - chooses a = arg max_{v∈V} Q̂(s, v) with probability 1 − ε_m
    Update (α̂, β̂) using MLE
    Store batch transitions in B
    for n = 1, ..., N do
        Sample minibatch of transitions (if λ > 0 collect consecutive states)
        Compute target values using CTD(λ) with target value network parameters
        Perform gradient step using loss MSE loss function
        Update target parameters
```

4.1 State and State-Action Value Functions

Let E_π denote expectation under the policy π and let $\tilde{\pi}$ denote the optimal policy, meaning for any state-dependent policy π and $s \in \mathcal{S}$ we have

$$E_\pi \left[\sum_{i=1}^{\infty} r_i \prod_{j=0}^{i-1} \gamma_j \middle| S_0 = s \right] \leq E_{\tilde{\pi}} \left[\sum_{i=1}^{\infty} r_i \prod_{j=0}^{i-1} \gamma_j \middle| S_0 = s \right].$$

The policy dependent value function is defined as

$$V_\pi(s) = E_\pi\left[\sum_{i=1}^{\infty} r_i \prod_{j=0}^{i-1} \gamma_j \middle| S_0 = s\right]$$

and the policy dependent state-action value function is defined as

$$Q_\pi(s,a) = E\left[r_1 + \gamma_1 V_\pi(S_1) | S_0 = s, a_0 = a\right].$$

Observe that the distribution of transitions $(a_i, S_i) \to (r_{i+1}, \gamma_{i+1}, S_{i+1})$ in this setting take a convenient form. Since (almost surely) at most one job arrival or service can happen at any moment in time, the state jumps $S_{i+1} - S_i$ take values in the space

$$\mathcal{T} \doteq \{e_j, -e_j : j \in \{1, ..., J\}\},$$

where e_j is the unit vector in $\mathbb{R}^J$ with jth entry 1 and all other entries 0. We have

$$P\left(S_{i+1} - S_i = e_j | S_i = s, a_i = a\right) = \frac{\alpha_j}{\nu(s,a)}$$

and

$$P\left(S_{i+1} - S_i = -e_j | S_i = s, a_i = a\right) = \frac{\beta_j a_j \mathcal{I}_{\{s_j > 0\}}}{\nu(s,a)}$$

for all j, where

$$\nu(s,a) = \sum_j \left(\alpha_j + \beta_j a_j \mathcal{I}_{\{s_j>0\}}\right).$$

Conditional on $(S_i = s, a_i = a)$, $\tau_{i+1} - \tau_i$ is exponential with rate $\nu(s,a)$. In addition, given (S_i, a_i), $S_{i+1} - S_i$ and $\tau_{i+1} - \tau_i$ are conditionally independent. Consequently

$$\begin{aligned} Q_\pi(s,a) &= E\left[\kappa^{-1}\left(1 - e^{-\kappa(\tau_1 - \tau_0)}\right)(-h(s)) + e^{-\kappa(\tau_1-\tau_0)} V_\pi(S_1) | S_0 = s, a_0 = a\right] \\ &= \frac{-h(s) + \sum_j \left(V_\pi(s+e_j)\alpha_j + V_\pi(s-e_j)\beta_j a_j \mathcal{I}_{\{s_j>0\}}\right)}{\kappa + \nu(s,a)}. \end{aligned} \tag{4.1}$$

We can use this specific form of $Q_\pi(s,a)$ to make estimating this state-action value function more efficient. In particular, the distribution of $(r_{i+1}, \gamma_{i+1}, S_{i+1} - S_i)$ is only impacted by S_i though which queue lengths are empty, and the dependence on the action is captured by the arrival and service rates (α, β). As a result, I use MLE estimates of (α, β) and an MLP estimate of $V_{\tilde{\pi}}(s)$ to create the estimate of $Q_{\tilde{\pi}}(s,a)$ using formula 4.1.

4.2 MLE Rate Estimates

Every transition provides information about (at least some of) the rates which allows us to estimate them efficiently. Assume we have N transition observations (not necessarily consecutive or from the same trajectory), $\{(s(n), a(n), \phi(n), \upsilon(n))\}_{n=1}^N$, where s is the initial state, a is the action, ϕ is the state change (i.e. $S_i - S_{i-1}$), and υ is the wait time until the state change (i.e. $\tau_i - \tau_{i-1}$). The specific form of the transition probabilities described in the previous section is used to compute the following MLE estimates. For all j let $u_j = \sum_{n=1}^N \mathcal{I}_{\{e_j\}}(\phi(n))$ be the number of unit increases in the jth queue length and $d_j = \sum_{n=1}^N \mathcal{I}_{\{-e_j\}}(\phi(n))$ be the number of unit decreases in the jth queue length. Then the MLE estimates for the rate parameters are

$$\hat{\alpha}_j = \frac{u_j}{\sum_{n=1}^N \upsilon(n)} \text{ and } \hat{\beta}_j = \frac{d_j}{\sum_{n=1}^N \upsilon(n) a_j(n) \mathcal{I}_{\{s_j(n)>0\}}} \text{ for all } j.$$

4.3 Action Space

Recall that we are working with a continuous action space defined by linear inequalities

$$\mathcal{A} \doteq \left\{ a \in \mathbb{R}_+^J : Ka \leq C \right\}.$$

One way to make searching this bounded, convex *polytope* more convenient is to compute its vertices $\mathcal{V} = \{v_1, ..., v_n\}$ offline and use the fact that the action space is the convex hull of this collection of vertices, i.e.

$$\mathcal{A} = \left\{ \sum_{k=1}^{n} w_k v_k : w_k \geq 0 \text{ for all } k \text{ and } \sum_{i=k}^{n} w_k = 1 \right\}.$$

Then instead we can focus on choosing good weights w (for instance, using a softmax function), and easily compute the corresponding action as $a = Vw$ where $V = \begin{bmatrix} v_1 & v_2 & \dots & v_n \end{bmatrix}$ is the $J \times n$ matrix whose columns are the vertices $\mathcal{V}$.

We can actually restrict our search to the vertices themselves (rather than a convex combination of them). This is because, due to the specific form of $Q_\pi(s, a)$ given in 4.1, for any $a_1, a_2 \in \mathcal{A}$ and $r \in [0, 1]$ we have

$$Q(s, ra_1 + (1 - r)a_2) = \rho(r)Q(a_1, q) + (1 - \rho(r))Q(a_2, q)$$

where

$$\rho(r) = \frac{r\nu(s, a_1) + r\kappa}{r\nu(s, a_1) + (1 - r)\nu(s, a_1) + \kappa}.$$

Since every $a \in \mathcal{A}$ is an convex combination of $v \in \mathcal{V}$. It follows that

$$\sup_{a \in \mathcal{A}} Q(s, a) = \max_{v \in \mathcal{V}} Q(s, v).$$

Consequently, outside of using exploratory actions with probability ϵ in the ϵ-greedy policy, I will treat $\mathcal{V} \subset \mathcal{A}$ as the action space.

4.4 Value Function Estimate

The goal is to estimate $Q_{\tilde{\pi}}(s, v)$ (recall we're only considering vertex actions $v \in \mathcal{V}$), and due to (4.1) and our MLE estimates of the rates what remains is to estimate $V_{\tilde{\pi}}(s)$. For this I use an MLP and denote this function estimate as $\hat{V}(s)$. I don't explicitly denote the parameter dependence in the notation to reduce clutter, but I write $\hat{V}^-(s)$ to denote the network based on the target parameters. The target parameters could be a direct copy of the regular parameters that isn't part of the computational graph (which is what is done in the examples) or separate values which are updated using a soft update. The parameters for $\hat{V}(s)$ are updated using stochastic gradient descent. The loss is the mean squared error between the prediction given by $\hat{V}$ and a target value estimate using $\hat{V}^-$. The target values are based on "conditional" n-step estimates which are described below.

Let $(\hat{\alpha}, \hat{\beta})$ denote the MLE estimates of (α, β),

$$\hat{Q}(s, a) = \frac{-h(s) + \sum_j \left(\hat{V}(s + e_j)\hat{\alpha}_j + \hat{V}(s - e_j)\hat{\beta}_j a_j \mathcal{I}_{\{s_j > 0\}} \right)}{\kappa + \hat{\nu}(s, a)} \text{ where } \hat{\nu}(s, a) = \sum_j \left(\hat{\alpha}_j + \hat{\beta}_j a_j \mathcal{I}_{\{s_j > 0\}} \right),$$

and

$$\hat{\pi}(s) \doteq \arg\max_{v \in \mathcal{V}} \hat{Q}(s, v).$$

Throughout this section I use the conventions that $\sum_{i=0}^{-1} ... = 0$ and $\prod_{j=0}^{-1} ... = 1$.

The conventional n-step value estimate of $V_{\tilde{\pi}}(s)$ is

$$\sum_{i=1}^{n} R_i \prod_{j=0}^{i-1} \gamma_j + \hat{V}^-(S_n) \prod_{j=0}^{n} \gamma_j$$

where $S_0 = s$ and the control $\hat{\pi}$ (the estimate of $\tilde{\pi}$) is used to choose the actions. Larger values of n result in estimates with less bias (less reliant on our value function estimates given by $\hat{V}^-$) but more variance (involve more random transitions). Recall that here the rate parameter estimates should converge relatively quickly (due to our ability to pool transition data from all states when estimating them) while estimating $V_{\tilde{\pi}}(s)$ is the more challenging task. As a result, I'd like to rely on these rate estimates when estimating $V_{\tilde{\pi}}(s)$ to reduce variance (while unfortunately adding some bias). The conditional expected value of the 1-step estimate is

$$E\left[R_0 + \gamma_0 \hat{V}^-(S_1) \middle| S_0 = s\right] = \frac{-h(s) + \sum_j \left(\hat{V}^-(s+e_j)\alpha_j + \sum_j \hat{V}^-(s-e_j)\beta_j \hat{\pi}_j(s)\mathcal{I}_{\{s_j>0\}}\right)}{\nu(s,\hat{\pi}(s)) + \kappa} \approx \tilde{V}^-(s)$$

where

$$\tilde{V}^-(s) \doteq \frac{-h(s) + \sum_j \left(\hat{V}^-(s+e_j)\hat{\alpha}_j + \sum_j \hat{V}^-(s-e_j)\hat{\beta}_j \hat{\pi}_j(s)\mathcal{I}_{\{s_j>0\}}\right)}{\hat{\nu}(s,\hat{\pi}(s)) + \kappa} \tag{4.2}$$

I define the conditional 1-step estimate as simply $\bar{V}^1(s) \doteq \tilde{V}^-(s)$. Note that for this estimate the state holding cost $h(s)$ is the relevant information provided by the transition, but the quantities that the depend on the action, $\hat{V}^-(S_1)$ and $e^{-\kappa\tau_1}$, are replaced with expectations. This means that even if a transition didn't use an action from the policy $\hat{\pi}$ it can still be used in this estimate.

For $n \geq 2$ we want the conditional n-step estimates to use subsequent trajectory information to improve the conditional 1-step estimate, while doing so in a way that the adjustments are efficient to implement. If we have a 2-step trajectory we now also have the conditional 1-step estimate for $V_{\tilde{\pi}}(S_1)$ given by $\tilde{V}^-(S_1)$. In order to use this in our estimate of $V_{\tilde{\pi}}(s)$ we need to account for the fact that S_1 is only one of the possible states next states used in our estimate $\tilde{V}^-(s)$. For example, if $S_1 = s + e_k$ then in (4.2) we need to replace $\hat{V}^-(s+e_k)$ with $\tilde{V}^-(S_1)$. Specifically, the conditional 2-step estimate of $V_{\tilde{\pi}}(s)$ is

$$\bar{V}^2(s) \doteq \tilde{V}^-(s) + \xi(S_1, s)\zeta(S_1)$$

where

$$\xi(\tilde{s}, s) \doteq \begin{cases} \frac{\hat{\alpha}_j}{\hat{\nu}(s,\hat{\pi}(s))+\kappa} & \text{if } \tilde{s} - s = e_j \\ \frac{\hat{\beta}_j \hat{\pi}_j(s)\mathcal{I}_{\{s_j>0\}}}{\hat{\nu}(s,\hat{\pi}(s))+\kappa} & \text{if } \tilde{s} - s = -e_j. \end{cases}$$

and

$$\zeta(s) \doteq \tilde{V}^-(s) - \hat{V}^-(s).$$

In general, for any $n \geq 1$ the conditional n-step estimate is

$$\bar{V}^n(s) \doteq \tilde{V}^-(s) + \sum_{i=1}^{n-1} \zeta(S_i) \prod_{j=0}^{i-1} \xi(S_{j+1}, S_j).$$

The intuition behind it is that the estimate adjustments from later states in the process are discounted based on their probability of occurring under the policy $\hat{\pi}$ using our rate estimates $(\hat{\alpha}, \hat{\beta})$. Also, observe that in this estimate future states can still provide useful information even if their corresponding action didn't come from the policy $\hat{\pi}$, as long as their probability of occurring under $\hat{\pi}$ isn't 0.

Let T denote the last step in the trajectory. Using $\lambda \in [0, 1)$ eligibility tracing the CTD(λ) estimate is

$$\bar{V}_\lambda(s) \doteq (1-\lambda) \sum_{i=1}^{T-1} \lambda^{i-1} \bar{V}^i(s) + (1-\lambda)\bar{V}^T(s) \sum_{l=T-1}^{\infty} \lambda^l$$

which can be written as

$$\bar{V}_\lambda(s) = \tilde{V}^-(s) + \sum_{i=1}^{T-1} \lambda^i \zeta(S_i) \prod_{j=0}^{i-1} \xi(S_{j+1}, S_j)$$

which is a more convenient form for implementation. Clearly the CTD(1) estimate is simply $\bar{V}_1(s) = \bar{V}^T(s)$.

5 Results

I tested the algorithm on three relatively simple networks which are discussed in Harrison, Mandayam, Shah, and Yang (note that there they consider ergodic cost as opposed to the discounted cost used here). Those authors introduce a control (action) policy called UFOS (utilization first, output second) which is based on the HGI order of priorities and approximates HGI performance well on these networks. I include UFOS as an additional performance benchmark which allows comparison of control choices (recall that HGI performance isn't based on a control scheme, but rather a lower bound for the workload process using (2.1)).

HGI prioritizes workload minimization (no wasted bandwidth) so it is asymptotically optimal when the holding cost is a monotonic function of the workload, i.e. $\tilde{w} \geq w$ (component-wise) implies $\hat{h}(\tilde{w}) \geq \hat{h}(w)$. When this monotonicity fails it may be possible to improve upon HGI and UFOS performance. For each of the three networks I consider two different linear holding costs. One (as in the paper referenced above) where all job types have a uniform holding cost of 1, and another where the *local traffic* job types (job types which require only a single resource for processing) have their holding cost increased to 4. This is done to "increase the nonmonoticity" of $\hat{h}(w)$, essentially making the decision between immediate holding cost reduction and maximizing bandwidth utilization more difficult.

In this section I discuss the flowrate ($x_j = \mathcal{I}_{\{s_j>0\}} a_j$) rather than action ($a_j$) assigned to job types because flowrate accounts for the fact that empty queues cannot receive processing rate. I use $r_c(x)$ to denote the holding cost reduction rate of flowrate x. I call a flowrate *viable* for a given queue length if it assigns 0 processing rate to all empty queues. The UFOS algorithm operates by assigning flowrates priorities and choosing the highest viable one for any given queue length.

The DRL results for all networks were obtained using the following hyperparameters:

- 64 units per layer, 2 hidden layers, and hyperbolic tangent activation functions used in the $\hat{V}$ MLP
- Target value parameters are just copies of regular value parameters removed from the computational graph (I didn't observe improved performance with soft update on target parameters)
- $M = 201$ batches
  - A batch consists of 1000 frames from each of 5 parallel environments
- $N_{\text{rand}} = 1$ random vertex batches
- $N_{\text{mb}} = 1000$ training minibatches per batch
  - A minibatch consists of 1000 frames
- ϵ_m decreases linearly in m from $\epsilon_2 = 1$ to $\epsilon_{201} = .1$
- Training environment cost bound of 150 when local cost is 1, and a training environment cost bound of 400 when local cost is 4
- Queues begin empty in all examples

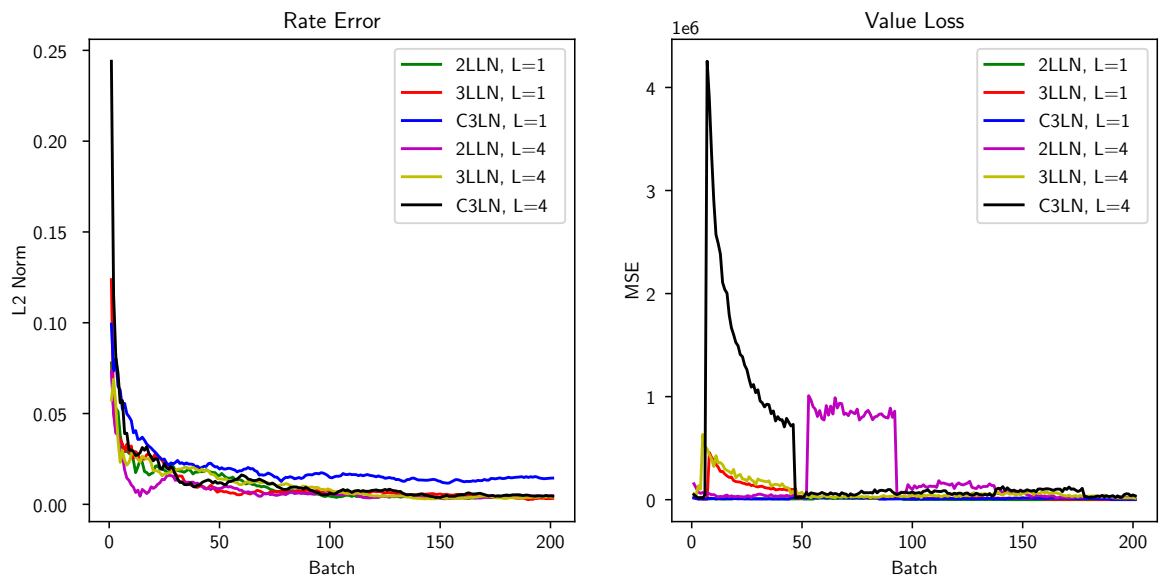

Figure 5.1: Rate Estimate Error and Average Value Loss for $\text{DRL}_{\lambda=1}$ Training

For additional hyperparameter values see the code.

In figure 5.1 the L^2 norm of the rate parameter estimate error and the average (per batch) minibatch value function MSE are displayed for the $\text{DRL}_{\lambda=1}$ training on all six networks. As expected, the rate estimates converge quickly. I tested the algorithm using $\lambda = 0, .5, 1$, but the performance was frequently similar so, as in figure 5.1, in some situations I will only present information from one of them. My intiution is that $\lambda > 0$ produces better value estimates, but the correlation that comes from requiring trajectories in the minibatches may slow training.

5.1 2LLN

I start with the *Two-Link Linear Network* (2LLN) discussed previously (for an illustration see 2.1). Recall there are $J = 3$ job types and $I = 2$ resources. The first two job types are *local traffic* (only use one resource). The third job type, which I'll refer to as *shared traffic*, uses both resources. The following parameters were used:

$$K = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, C = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \beta = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \alpha = \begin{bmatrix} .4 \\ .4 \\ .4 \end{bmatrix}, \kappa = .01$$

L stands for the linear holding cost on local traffic jobs ($j = 1, 2$). The queue length holding cost and corresponding workload holding cost are

$$h(s) = L(s_1 + s_2) + s_3 \text{ and } \hat{h}(w) = L(w_1 + w_2) - (2L - 1)w_1 \wedge w_2$$

where the *uniform* holding cost example uses $L = 1$ and the *increased local* holding cost example uses $L = 4$.

The UFOS algorithm uses the following flowrate priorities, moving down the list until it reaches a flowrate that is *viable* (meaning it assigns 0 flowrate to all empty queues). For instance, if $s = [0, 2, 2]$ then $x = [1, 1, 0]$ is not viable (because queue 1 is empty) so $x = [0, 0, 1]$ is chosen. Recall that $r_c(x)$

indicates the rate of holding cost reduction for flowrate x (when it is viable).

Algorithm 2 2LLN UFOS Flowrate Priorities

1. $x = [1, 1, 0] \to r_c(x) = 2L$
2. $x = [0, 0, 1] \to r_c(x) = 1$
3. $x = [1, 0, 0] \to r_c(x) = L$ or $x = [1, 0, 0] \to r_c(x) = L$

With the *uniform* holding cost $\hat{h}(w)$ is monotonic but with the*increased local* cost it is not, meaning that in some cases you can increase the rate of holding cost reduction by wasting bandwidth. Recall that UFOS always prioritizes bandwidth utilization even when such opportunities exist. For instance, in the $L = 4$ example if the shared traffic queue and one (but not both) local traffic queues are nonempty (for instance, $s = [0, 2, 2]$) UFOS will process the shared job rather than the local job to avoid wasting bandwidth ($x = [0, 1, 0]$ would idle resource 1 while $x = [0, 0, 1]$ has them both at full capacity). However, processing the local job instead has a holding cost reduction rate benefit of 3 ($r_c([0, 1, 0]) - r_c([0, 0, 1]) = 3$). Note that this is an example of *blocking*. By processing the only available local traffic job, one of the resources must be idle despite having a positive workload from the shared traffic.

Table 5.1 shows the performance of this algorithm (DRL) compared to the two benchmarks. Note that these results are reported in terms of cost rather than negative reward so smaller numbers correspond to better performance.

Table 5.1: 2LLN Total Cost Results

Cost	HGI	UFOS	$\text{DRL}_{\lambda=0}$	$\text{DRL}_{\lambda=.5}$	$\text{DRL}_{\lambda=1}$
Uniform	470.95	502.24	493.90	506.90	517.72
Increased Local	1350.5	1510.4	1229.0	1224.3	1260.6

For the 2LLN the only nontrivial choice, in the sense that processing one job type prevents the processing of another, is between local traffic and shared traffic when both are available (meaning the shared traffic queue is nonempty and at least one local traffic queue is nonempty). In table 5.2 I provided the average flowrate the $\text{DRL}_{\lambda=0}$ algorithm assigned in these situations to compare with the UFOS priorities described above (outside of these situations the $\text{DRL}_{\lambda=0}$ flowrates are almost always identical to the UFOS flowrates). In this table the *Local Traffic Available* column indicates how many of the local traffic queues are nonemtpy, the *Local Cost Benefit* column provides the increase in holding cost reduction rate (r_c) from processing all available local traffic instead the shared traffic, and the *Local Bandwidth Waste* column provides the total capacity underutilization of all resources due to processing local traffic. Note that, while the $\text{DRL}_{\lambda=0}$ policy is deterministic, it is a function of the queue length rather than simply which are queues empty so this is an average (under the occupation measure) over the queue lengths that fall into each category.

Table 5.2: 2LLN Average Flowrate: Local vs Shared Traffic

Holding Cost	Local Traffic Available	Local Cost Benefit	Local Bandwidth Waste	$\text{DRL}_{\lambda=0}$ Flowrate		UFOS Flowrate	
				Local	Shared	Local	Shared
Uniform	All	1	0	.847	.153	1	0
	One	0	1	0	1	0	1
Increased Local	All	7	0	.998	0	1	0
	One	3	1	.400	.600	0	1

There is a clearly correct choice whenever there is a cost benefit with no bandwidth waste, or bandwidth waste with no cost benefit. We see that in the *uniform* holding cost example $\text{DRL}_{\lambda=0}$, aside from some small inefficiencies when all local traffic jobs are available, has similar priorities to UFOS which allows it to achieve similar performance. The ability of $\text{DRL}_{\lambda=0}$ to outperform both the UFOS and HGI benchmarks in the *increased local* holding cost example appears to be due to its willingness to sacrifice bandwidth utilization for the cost benefit about 40% of the time by processing local traffic and when only one local traffic job is available.

5.2 3LLN

The *Three-Link Linear Network* (3LLN) consists of $J = 4$ job types and $I = 3$ resources. The structure is similar to 2LLN in that the last job type requires all resources for processing (the shared traffic job) and the first three are local traffic. As a result, it lends itself to similar analysis. The following parameters are used:

$$K = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}, C = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \beta = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \alpha = \begin{bmatrix} .4 \\ .4 \\ .4 \\ .4 \end{bmatrix}, \kappa = .01$$

The queue length holding cost and corresponding workload holding cost are

$$h(s) = L(s_1 + s_2 + s_3) + s_4 \text{ and } \hat{h}(w) = L(w_1 + w_2 + w_3) - (3L - 1)w_1 \wedge w_2 \wedge w_3$$

where again L stands for the holding cost on local jobs, the *uniform* holding cost example uses $L = 1$, and the *increased local* holding cost example uses $L = 4$. $\hat{h}(w)$ is nonmonotonic both when $L = 1$ and $L = 4$, but when $L = 4$ and there are more opportunities to increase the rate of holding cost reduction by wasting bandwidth.

The UFOS algorithm uses the following flowrate priorities, moving down the list until it reaches a flowrate that is *viable*.

Algorithm 3 3LLN UFOS Flowrate Priorities

1. $x = [1, 1, 1, 0] \to r_c(x) = 3L$
2. $x = [0, 0, 0, 1] \to r_c(x) = 1$
3. $x = [1, 1, 0, 0] \to r_c(x) = 2L$, $x = [1, 0, 1, 0] \to r_c(x) = 2L$, or $x = [0, 1, 1, 0] \to r_c(x) = 2L$
4. $x = [1, 0, 0, 0] \to r_c(x) = L$, $x = [0, 1, 0, 0] \to r_c(x) = L$, or $x = [0, 0, 1, 0] \to r_c(x) = L$

Table 5.3 contains the performance results for the 3LLN network. Note that DRL slightly outperforms UFOS when $L = 1$, but has a greater performance advantage (and beats the HGI benchmark) when $L = 4$ and the nonmonotonicty of $\hat{h}(w)$ is more severe.

Table 5.3: 3LLN Total Cost Results

Cost	HGI	UFOS	$\text{DRL}_{\lambda=0}$	$\text{DRL}_{\lambda=.5}$	$\text{DRL}_{\lambda=1}$
Uniform	687.56	741.57	699.96	733.92	735.02
Increased Local	2384.6	2591.1	1862.5	2139.6	1864.2

Just like in 2LLN, the only nontrivial choice is between local traffic and shared traffic when both are available. In table 5.4 I provided the average flowrate the $\text{DRL}_{\lambda=1}$ algorithm assigned in these situations to compare with the UFOS priorities described above. In this table the *Local Traffic Available* column indicates how many of the local traffic queues are nonemtpy, the *Local Cost Benefit* column provides the increase in holding cost reduction rate (r_c) from processing all available local

traffic instead the shared traffic, and the *Local Bandwidth Waste* column provides the total capacity underutilization of all resources due to processing local traffic.

Table 5.4: 3LLN Average Flowrate: Local Traffic vs Shared

Cost Type	Local Traffic Available	Local Cost Benefit Benefit	Local Bandwidth Waste	$\text{DRL}_{\lambda=1}$ Flowrate		UFOS Flowrate	
				Local	Shared	Local	Shared
Uniform	All	2	0	.816	.184	1	0
	Two	1	1	.410	.590	0	1
	One	0	2	.029	971	0	1
Increased Local	All	11	0	1	0	1	0
	Two	7	1	1	0	0	1
	One	3	2	.034	.966	0	1

While there is some inefficiency present in $\text{DRL}_{\lambda=1}$ as indicated by the first row, it has a performance advantage due to its willingness to sometimes waste bandwidth to increase holding cost reduction. As you'd expect, in both the *uniform* and *increased local* examples, the $\text{DRL}_{\lambda=1}$ algorithm is more willing to underutilize resource capacity to increase the rate of holding cost reduction when the bandwidth waste is smaller and the cost benefits are greater.

5.3 C3LN

The *Complex Three-Link Network* (C3LN) consists of $J = 6$ job types and $I = 3$ resources. The incidence matrix and parameter values used are as follows:

$$K = \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}, C = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \beta = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \alpha = \begin{bmatrix} .32 \\ .16 \\ .4 \\ .24 \\ .16 \\ .24 \end{bmatrix}, \kappa = 0.1$$

The queue length holding cost is

$$h(s) = L(s_1 + s_2 + s_3) + s_4 + s_5 + s_6$$

and corresponding workload holding cost is

$$\hat{h}(w) = L(w_1 + w_2 + w_3) - (L-1)w_1 \wedge w_2 - (L-1)w_2 \wedge w_3 - w_1 \wedge w_2 \wedge w_3 - L(w_1 \wedge w_2) \vee (w_2 \wedge w_3).$$

L stands for the holding cost on local jobs, with the *uniform* holding cost example using $L = 1$ and the *increased local* holding cost example using $L = 4$. $\hat{h}(w)$ is already nonmonotonic both when $L = 1$ and this is more extreme when $L = 4$.

Due to the number of job types, the UFOS flowrate priority list for C3LN is long (it can be found in the code) so instead I'll just summarize the priority logic. UFOS chooses a flowrate x which maximizes the holding cost rate reduction, among all viable *bandwidth maximizing* flowrates. In other words, UFOS first restricts the search to flowrate which don't waste bandwidth, then chooses one which maximizes $r_c(x)$.

Table 5.5 contains the performance results for the C3LN. Although $\hat{h}(w)$ is nonmonotonic even in the *uniform* holding cost example, the DRL performance is still slighly worse than UFOS. Once this nonmonotonicity is increased in the *increased local* example DRL outperforms UFOS as well as the HGI benchmark. Although the advantage of DRL in the *increased local* holding cost example is surely

due to wasting bandwidth in favor of increased holding cost reduction in certain situations, the number choices is greater here than it was in the 2LLN and 3LLN situations so the average flowrate analysis has been omitted.

Table 5.5: C3LN Total Cost Results

Cost	HGI	UFOS	$DRL_{\lambda=0}$	$DRL_{\lambda=.5}$	$DRL_{\lambda=1}$
Uniform	610.38	665.75	687.77	686.64	692.71
Increased Local	1611.6	1918.8	1569.3	1452.9	1467.5